

# Technische Dokumentation Kühlschrankmanager KM3003

Samuel Brunner

November 12, 2024

# Contents

|          |                                                  |          |
|----------|--------------------------------------------------|----------|
| <b>1</b> | <b>Einleitung</b>                                | <b>3</b> |
| <b>2</b> | <b>Plattform und Systemarchitektur</b>           | <b>3</b> |
| <b>3</b> | <b>Software</b>                                  | <b>4</b> |
| 3.1      | Verwendete Bibliotheken und Frameworks . . . . . | 4        |
| 3.2      | Datenbankstruktur . . . . .                      | 5        |
| 3.3      | Testing . . . . .                                | 5        |
| <b>4</b> | <b>Deployment</b>                                | <b>5</b> |
| <b>5</b> | <b>Herausforderung bei der Implementierung</b>   | <b>5</b> |
| <b>6</b> | <b>Ausblick und Verbesserungsvorschläge</b>      | <b>5</b> |
| <b>7</b> | <b>Inline documentation anhängen</b>             | <b>6</b> |

# 1 Einleitung

Der Kühltischmanager im Vertretungszimmer der Fachschaft ersetzt die Strichliste, welche früher am Kühltisch hing, und die Getränkekäufe der Vereinsmitglieder erfasste.

Früher wurden auf der Liste für jedes erworbene Getränk Striche in der Zeile des jeweiligen Mitglieds und in der Spalte des Getränks eingetragen. Regelmäßig zählte man die Striche, ermittelte so die Saldos der Mitglieder und erstellte eine neue Liste.

Der KM3003 ist die Weiterentwicklung der bisherigen Kühltischmanager (KM3000 bis KM3002). Der KM3002 basierte auf einem Raspberry Pi mit Touchscreen und NFC-Reader in einem 3D-gedruckten Gehäuse. Mitglieder wurden per Studentenausweis mit NFC identifiziert, und Käufe über den Touchscreen ausgewählt. Die Daten wurden in einer MySQL-Datenbank gespeichert, die jedoch nur negative Salden ermöglichte und so exakte Abrechnungen erzwang. Zudem gab es Verbesserungsbedarf in Codequalität, Zuverlässigkeit und Geschwindigkeit.

Der KM3003 erlaubt nun die Identifizierung von Mitgliedern und Produkten über das Scannen von Barcodes. Mitglieder werden über die Barcodes auf den Studentenausweisen erkannt, Produkte über EAN-Codes. Außerdem erlaubt er nun auch positive Saldos, sodass Mitglieder flexibel Guthaben einzahlen können.

Zusammengefasst ist der KM3003 ein Self-Checkout Point-of-Sale-Gerät mit Touchscreen und Barcode-Scanner.

# 2 Plattform und Systemarchitektur

Als Basis für den Kühltischmanager wurde ein Microsoft Surface 3 mit dem zugehörigen Dock verwendet. Das Surface 3 ist ein Tablet-Computer mit einem zuverlässigen kapazitiven Multi-Touchscreen und solider Treiberunterstützung unter Windows. Als Betriebssystem wird ein unverändertes Windows 11 Home verwendet, um eine gut getestete und stabile Basis zu bieten. Auch die Verfügbarkeit von Updates in der Zukunft wird so maximiert.

Weiterhin bieten die meisten Microsoft Surface Geräte einen Kioskmodus der spezielle Lösungen für Geräte mit festem und ortsunveränderlichem Verwendungszweck bietet. So optimiert der Kioskmodus das Ladeverhalten für Geräte, die permanent mit Strom versorgt werden, um die Akkulebensdauer

zu verlängern. Weiterhin kann der Zugang zu Anwendungen, die nicht dem gedachten Verwendungszweck entsprechen, beschränkt werden.

Zum scannen wird ein Freihandscanner (Modell DS9208) der Firma Zebra verwendet. Dieser wird per USB angeschlossen, meldet sich aber unter Windows, als ein über serielle Schnittstelle angebundenes Gerät. Dies erfolgt unter Verwendung des USB communications device class (USB CDC) Treibers.

Das Surface speichert keinerlei Daten. Hierfür wird eine externe MySQL-Datenbank verwendet, welche Nutzer, Produkte, Ein- bzw. Auszahlungen und Einkäufe enthält.

### 3 Software

Als Implementierungssprache wurde objektorientiertes Python3 verwendet. Python ist weit verbreitet und einfach zu lernen, was Wartungsarbeiten oder die Erweiterung mit neuen Features für zukünftige Mitglieder erleichtert.

Weiterhin können Python Anwendungen auf Linux und Windows Plattformen ausgeführt werden, sodass bei Bedarf auch das bisher verwendete Raspberry Pi mit externem Touchscreen weiter verwendet werden könnte.

Auch die Verfügbarkeit von bestehenden Bibliotheken für graphical user interface (GUI), Datenbankbindung und serielle Schnittstellen ist optimal.

#### 3.1 Verwendete Bibliotheken und Frameworks

- Lizenzmodell PySimpleGUI; PySimpleGUI license \* "Hobbyist Developer" means any individual who uses PySimpleGUI for development purposes solely for either or both of the following: (1) personal (e.g., not on behalf of an employer or other third party), Non-Commercial purposes; or (2) Non-Commercial educational or learning purposes (1 and 2 together, the "Permitted No-cost Purposes"). Lizenz ist aktuell so und erlaubt uns die Nutzung, weil wir keinen Gewinn machen. Ziel der Kasse ist 0. Bleibt das so? Die Andere Option wäre \$99 zu zahlen. (Jährlich um auch die zukünftigen Releases zu nutzen). Also eigtl \$99 pro Jahr tkinter: Tcl/Tk license \* The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included

verbatim in any distributions. Hat sich laut wayback machine auch seit 2010 nicht geändert

- Dokumentation: PySimpleGUI: extrem auf den Hobbyist ausgelegt, sehr beipiellastig meiner meinung nach furchtbar zu lesen hätte lieber eine ordentliche API spezifikation, als 10000 beispielprogramme tkinter: standard doku weniger an den Hobbyisten gerichtet; für den geübten aber einfacher das zu finden was man sucht (Werkzeuge, wie configure() )

- Architektur und Objekte: tkinter: everything is a widget widgets are organized in a hierarchy widgets have configuration options the placement options is managed by a geometry manager the GUI reacts to user inputs and changes from my program and refreshes the interface

- pysimplegui: everything

- Lernkurve:

- mysql connector

## **3.2 Datenbankstruktur**

TODO an Normalformen angelehnt atomare Felder

## **3.3 Testing**

Unit tests schreiben mit pyunit test Framework

## **4 Deployment**

- deployment: surface, ansible for km3003 standalone cyclical restart windows task scheduler mysql db

## **5 Herausforderung bei der Implementierung**

- wiederaufbau von Verbindungsabbrüchen (Datenbank und Scanner) zur Laufzeit

## **6 Ausblick und Verbesserungsvorschläge**

UI Framework is kacke Sounds

## 7 Inline documetation anhängen

**USB CDC** USB communications device class

**GUI** graphical user interface

## List of Figures

## References

- [1] Microsoft Kiosk Mode <https://learn.microsoft.com/en-us/windows/iot/iot-enterprise/customize/kiosk-mode>
- [2] What is infrastructure as code (IaC)? <https://learn.microsoft.com/en-us/devops/deliver/what-is-infrastructure-as-code>
- [3] tkinter — Python interface to Tcl/Tk <https://docs.python.org/3/library/tkinter.html>